

Qiskit Fall Fest Hackathon: Problem Statements

Oct 9, 2026 · @Vatsal

Read this first

You have one week, a laptop, and AI assistants. The ten problem statements below are **suggestions, not a menu you must order from**. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

What we reward. A project that works, that you can explain line by line, and that is honestly compared against a classical alternative. Using AI to write code is allowed and encouraged; being unable to explain or verify that code is not.

How each statement is laid out. Every problem has the same parts: a short background, the problem itself, the core tasks (what a complete submission does), stretch goals (what makes it excellent), and a hint on tools. Treat the core tasks as the minimum, not the ceiling.

Simulators first, hardware as a bonus. Everything here can be done on a simulator (Qiskit Aer or the statevector tools). Running a small version on real IBM hardware, with a clear comparison against the simulator, earns extra credit in any track.

Mix and match. The strongest projects will connect tracks. A few examples:

- Encode a logical qubit with the Steane code, then use it inside a QKD protocol or an optimisation algorithm.
- Train a quantum neural network to act as a decoder or error corrector.
- Use LDPC decoding for the error-correction (reconciliation) step of a QKD protocol.

More combinations are listed in the "Go beyond" section near the end.

Track 1: Quantum chemistry

Problem 1: The hydrogen molecule with VQE

Background. Hydrogen (H_2) is the smallest molecule where electrons genuinely interact, and the standard first test for quantum chemistry on quantum computers. Its ground-state energy depends on the distance between the two atoms, and tracing that

dependence gives the bond curve that tells chemists how strong the bond is and where it sits.

Problem. Compute the ground-state energy of H_2 across a range of bond lengths using the Variational Quantum Eigensolver, and decide how trustworthy your answer is.

Core tasks.

- Build the molecular Hamiltonian for H_2 (minimal basis, for example STO-3G) and map it to qubits. Reduce it to as few qubits as you can using symmetries, and say how.
- Implement VQE with at least two different ansatz circuits (for example a chemistry-inspired one and a hardware-efficient one) and a classical optimiser of your choice.
- Plot energy against bond length from about 0.3 to 3 Å, and mark the equilibrium bond length and the dissociation limit.
- Compare against exact diagonalisation of the same Hamiltonian. Report the energy error in Hartree and say whether it is below chemical accuracy (about 1.6 milliHartree).

Stretch goals.

- Add a noise model and show how error grows, then apply an error mitigation technique and measure the improvement.
- Run one bond length on real IBM hardware and compare it with the simulator.
- Compute an excited state (for example with a variational method such as VQD) or the molecular dipole.
- Report how the number of circuit evaluations and the optimiser choice affect convergence.

Hints. Qiskit Nature with PySCF builds the Hamiltonian; the Estimator primitive evaluates energies. A good submission explains why the ansatz is or is not expressive enough, not only that the numbers match.

Problem 2: Cyclohexane with sample-based quantum diagonalisation (SQD)

Background. Cyclohexane (C_6H_{12}) is a ring that can fold into several shapes (conformers), such as the chair, the boat and the twist-boat. The chair is the most stable, and the energy gaps between shapes decide how the molecule behaves. A molecule this size is far beyond what plain VQE can handle on any current device. Sample-based quantum diagonalisation takes a different route: a quantum computer samples important electron configurations, and a classical computer then solves the Hamiltonian restricted to those configurations.

Problem. Estimate the relative energies of cyclohexane conformers using SQD, and measure how well the sampled subspace captures the true ground state.

Core tasks.

- Choose a manageable active space (a handful of the most relevant orbitals and electrons) and justify the choice. Generate geometries for the chair and at least one other conformer.
- Prepare a sampling circuit (for example a local unitary cluster Jastrow ansatz), sample bit strings, and run the SQD workflow, including the configuration recovery step that repairs samples corrupted by noise.
- Report the energy of each conformer and the energy differences between them, in kcal/mol.
- Compare against a classical reference (such as a CASCI calculation in PySCF, or a larger classical method) and report the error.

Stretch goals.

- Study convergence: plot the energy against subspace size and against the number of samples.
- Compare samples from an ideal simulator with samples from a noisy backend or real hardware.
- Identify which conformer ordering is reproduced and which is not, and explain why.
- Explore larger active spaces and report where the classical step becomes the bottleneck.

Hints. IBM provides the open-source `qiskit-addon-sqd` package and tutorials; `ffsim` simulates the fermionic sampling circuits. Remember that you are expected to verify the literature ordering of the conformers rather than assume it. The honest question to answer is: how much of the accuracy comes from the quantum sampling and how much from the classical diagonalisation?

Track 2: Quantum machine learning

Problem 3: Build and benchmark a variational quantum classifier

Background. A variational quantum classifier encodes data into a quantum circuit, applies trainable gates, measures, and updates the gate angles to reduce a loss. Whether this ever beats classical models is an open research question, so the interesting work is a fair, careful test.

Problem. Build a VQC from scratch for a classification task of your choice, and find out when it helps, when it fails, and why.

Core tasks.

- Pick a dataset (for example a reduced version of Iris, a two-class slice of a standard dataset, or a synthetic one that is not linearly separable). Explain how you reduced it to fit a few qubits.
- Implement the three parts yourself: a feature map, a trainable ansatz and a readout. Train with a classical optimiser.
- Report training and test accuracy and show a learning curve. Use a held-out test set.
- Compare against at least two classical baselines with a similar number of parameters (for example logistic regression and a small neural network).

Stretch goals.

- Study the choice of feature map and ansatz depth, and show which design choices matter most.
- Compare the VQC with a quantum kernel method.
- Test robustness to shot noise and to a hardware noise model.
- Visualise the decision boundary in two dimensions.

Hints. Qiskit Machine Learning has ready-made VQC and kernel classes, but you are expected to open the box: show the circuit and explain each gate. An entry that reports "no advantage, and here is why" is valued over one that hides weak results.

Problem 4: A quantum neural network that avoids barren plateaus

Background. Deep, randomly initialised quantum circuits suffer from barren plateaus: the gradients shrink exponentially with the number of qubits, so training stalls. This is one of the main obstacles to scaling quantum neural networks. Several mitigation strategies exist, including starting from a near-identity circuit, training layer by layer, using local rather than global cost functions, and structured ansatz designs.

Problem. Show a barren plateau, then build and evaluate a quantum neural network that mitigates it.

Core tasks.

- Measure how the variance of a gradient component scales with the number of qubits (for example 2 to 10) and the circuit depth, for a randomly initialised hardware-efficient ansatz. Plot it on a log scale.
- Implement at least one mitigation strategy and repeat the measurement, showing that the gradient variance decays more slowly.
- Train your mitigated network on a small task (classification or regression) and compare the final loss and convergence speed with the unmitigated version.
- Explain, in words and with a plot, why your strategy works.

Stretch goals.

- Compare two or three strategies on the same task (for example identity-block initialisation, layer-wise training and a local cost).
- Show that a global cost function causes a plateau even for a shallow circuit.
- Investigate whether noise creates a plateau of its own, and what that means for hardware.
- Combine with another track, such as a quantum neural network that learns to decode or correct errors (see "Go beyond").

Hints. Use the parameter-shift rule or automatic differentiation on a simulator to get gradients. Average over many random initialisations; a single run tells you nothing about a variance.

Track 3: QAOA and optimisation

Problem 5: MaxCut with QAOA, and how good is good enough

Background. MaxCut asks you to split the dots of a network into two groups so that as many links as possible run between the groups. It models problems such as clustering, circuit layout and scheduling, and it is the standard test problem for QAOA.

Problem. Solve MaxCut on a family of graphs with QAOA and measure how the quality of the answer depends on the number of layers, the graph and the optimiser.

Core tasks.

- Build the cost operator for a graph and the QAOA circuit with p layers. Do it yourself rather than only calling a ready-made solver.
- Solve at least three graph types (for example a ring, a random graph and a 3-regular graph) with 6 to 12 nodes, for $p = 1, 2$ and 3.
- Report the approximation ratio (the cut you found divided by the best possible cut, from brute force) and the probability of sampling the optimal cut.
- Compare with a classical method: a greedy heuristic and the Goemans–Williamson approach, or at least a simple local search.

Stretch goals.

- Study the optimiser: compare COBYLA, SPSA and gradient-based methods, and test smart initial angles against random starts.
- Explore warm-starting QAOA from a classical solution, or adding a different mixer.
- Add a noise model and find the largest p that still helps.
- Run one small graph on real hardware.

Hints. Qiskit Optimization turns a graph problem into a cost Hamiltonian; the Estimator and Sampler primitives run the loop. Always check your result against brute force on small graphs before trusting it on larger ones.

Problem 6: A routing problem: deliveries on a small map

Background. Routing questions, such as the best order to visit a set of places or how to split deliveries between two vehicles, are central to logistics. They are hard because the number of possible routes explodes as places are added. Encoding them for a quantum computer is itself a skill: rules such as "visit every place exactly once" become penalty terms in the cost function.

Problem. Encode a small routing problem as a QUBO or Ising problem, solve it with QAOA, and find where the approach breaks down.

Core tasks.

- Choose a travelling-salesperson problem with 4 or 5 locations, or a vehicle-routing problem with a few locations and two vehicles. State the distance matrix.
- Write down the encoding: which qubit means what, the distance term, and the penalty terms that force a valid route. Count the qubits you need.
- Solve with QAOA and report: how often a valid route appears, how often the best valid route appears, and the cost of the best route found.
- Compare against brute force and against a classical heuristic, and discuss the effect of the penalty weight.

Stretch goals.

- Use a more qubit-efficient encoding and compare the results.
- Study how the fraction of invalid routes changes with the penalty weight and the number of layers.
- Scale to the largest instance your simulator can handle and plot runtime against problem size.
- Replace the plain mixer with one that keeps solutions valid, and show the improvement.

Hints. The qubit count grows quickly (roughly with the square of the number of locations), so keep instances small and focus on a clean encoding and an honest comparison. Do not claim a quantum speedup; show what you actually measured.

Track 4: Quantum error correction

Problem 7: A logical qubit with the Steane code

Background. The Steane code protects one logical qubit using seven physical qubits, and corrects any single error of any type (bit flip, phase flip or both). It is built from the classical Hamming code, which makes it a perfect first example of how classical ideas become quantum ones. Its structure also allows several logical gates to be applied transversally, meaning one gate per physical qubit, so a single faulty gate cannot spread through the block.

Problem. Build, test and use a working Steane-code logical qubit in Qiskit, and show that it really does protect information.

Core tasks.

- Build the encoding circuit that turns one qubit into the seven-qubit logical state, and draw it in the Qiskit Composer or with Qiskit's circuit drawer. Explain the stabilisers it uses.
- Build the syndrome extraction circuit with ancilla qubits, and the classical lookup that maps each syndrome to a correction.
- Inject each of the 21 possible single-qubit errors (X, Y or Z on each of the 7 qubits) and show that every one is detected and corrected. Present this as a table.
- Demonstrate at least one transversal logical gate (for example logical X, Z, H or CNOT between two blocks) and check it on the logical states.

Stretch goals.

- Show what happens with two errors: the code should fail, so measure the logical error rate against the physical error rate with a noise model and find the break-even point.
- Use mid-circuit measurement and feed-forward so the correction runs inside the circuit.
- Add flag qubits or repeated syndrome measurements, so errors in the syndrome circuit itself are not fatal.
- Use your logical qubit in a larger task (see "Go beyond"), such as a protected QKD or Grover step.

Hints. Qiskit's `if_test` handles classically controlled corrections, and the Aer simulator supports noise models. Keep a clear table of the stabiliser generators; most mistakes come from mixing up which qubits each one touches.

Problem 8: LDPC decoding from scratch

Background. Low-density parity-check (LDPC) codes use a sparse parity-check matrix, and are decoded by passing messages along a graph (the Tanner graph) until the nodes agree. They power modern communications such as 5G and Wi-Fi, they are used to correct errors in QKD key generation, and their quantum cousins are a leading route to practical, low-overhead error correction.

Problem. Implement an LDPC decoder, measure its performance against noise, and understand why it works and where it fails.

Core tasks.

- Construct a sparse parity-check matrix (for example a regular LDPC code with a few hundred bits), and draw a small example as a Tanner graph.
- Implement belief propagation decoding (the sum-product or min-sum variant) for bits sent through a binary symmetric channel.
- Plot the frame error rate and bit error rate against the channel error probability, and identify the threshold region where decoding starts to fail.
- Explain what the decoder is doing in one clear figure, such as how the messages evolve over iterations on a failing and a succeeding example.

Stretch goals.

- Compare belief propagation with a simpler decoder (bit flipping) and with an improved one, and report the difference.
- Move to quantum codes: build a hypergraph-product or bivariate bicycle style code, and decode with belief propagation combined with ordered statistics decoding (BP+OSD). Why does plain belief propagation struggle on quantum codes?
- Implement the decoder in a hardware-friendly way, for example fixed-point arithmetic, and report the cost of lower precision.
- Use your decoder inside the reconciliation step of a QKD protocol (see "Go beyond").

Hints. The open-source `ldpc` Python package can serve as a reference to check your own implementation, and `stim` simulates quantum error correction circuits efficiently. A decoder you wrote yourself and verified against a reference is worth more than one imported in a single line.

Track 5: Quantum key distribution

Problem 9: BB84 and BBM92 in Qiskit

Background. Quantum key distribution lets two people, Alice and Bob, agree on a secret key whose secrecy is guaranteed by physics: an eavesdropper who measures the qubits disturbs them, and the disturbance shows up as errors. In BB84, Alice sends single qubits prepared in randomly chosen bases. In BBM92, a source distributes entangled pairs and Alice and Bob each measure their half in randomly chosen bases; the entanglement does the job of the preparation.

Problem. Implement both protocols end to end in Qiskit, attack them with an eavesdropper, and show how secure key is extracted from noisy, partly compromised data.

Core tasks.

- Implement BB84: random bits and bases for Alice, random measurement bases for Bob, and the sifting step that keeps only matching bases. Report the sifted key rate (about half of the qubits in the ideal case).
- Implement BBM92 with Bell pairs and the same sifting logic. Show that it gives the same kind of key by a different route.
- Add an intercept-and-resend eavesdropper (Eve) and show that she raises the error rate in the sifted key (the quantum bit error rate, or QBER) to around 25 percent when she attacks every qubit. Plot QBER against the fraction of qubits Eve attacks.
- Add a channel noise model and a decision rule: when is the QBER low enough to keep the key, and when must the run be aborted?

Stretch goals.

- Implement error estimation on a sampled subset, error correction (reconciliation) and privacy amplification, so the output is a shorter final key. Estimate the final key rate against QBER.
- For BBM92, run a Bell-inequality (CHSH) test as a second security check and show that Eve's attack lowers the violation.
- Model losses and imperfect detectors, and study the distance dependence of the key rate.
- Replace reconciliation by LDPC decoding built in Problem 8, or protect the transmission with a Steane-coded logical qubit from Problem 7.

Hints. Simulating qubit by qubit with Aer is fine for a few thousand qubits; for larger runs, you can simulate the statistics directly. Be clear in your write-up about what is a physical simulation and what is a shortcut. A security argument that you can state in two

sentences is part of the deliverable.

Track 6: Post-quantum cryptography

Problem 10: Lattice-based cryptography from scratch

Background. Large quantum computers running Shor's algorithm would break the public-key systems used on the internet today, such as RSA and elliptic curve cryptography. Post-quantum cryptography replaces them with problems that are believed to be hard even for quantum computers. The leading family is based on lattices, and the problem called Learning With Errors (LWE): recover a secret from noisy linear equations. The new NIST standard for key exchange (ML-KEM, derived from Kyber) is a structured, efficient version of this idea.

Problem. Implement a small lattice-based encryption scheme, test its correctness, and explore where its security comes from.

Core tasks.

- Implement a toy LWE-based public-key encryption scheme (key generation, encryption of a bit or short message, decryption). Choose small parameters and write them down.
- Measure the decryption failure rate as you vary the noise, and explain the trade-off between noise (security) and correctness.
- Demonstrate an attack on deliberately tiny parameters, for example by lattice reduction (LLL) or by brute force, and show how the attack becomes infeasible as the dimension grows. Plot the cost against the dimension.
- Explain, in plain words and with one figure, why adding noise turns an easy linear-algebra problem into a hard one.

Stretch goals.

- Move to the structured version: implement polynomial arithmetic with the number theoretic transform and build a simplified Kyber-style key exchange.
- Contrast with the threat: run Shor's algorithm in Qiskit to factor a small number such as 15 or 21, and relate the resource needs of breaking real keys to what hardware exists.
- Compare key and ciphertext sizes with RSA and elliptic curve schemes.
- Examine one implementation weakness, such as a timing leak in your own code, and fix it.

Hints. This track is mostly classical code, so the quantum link is the story you tell: why Shor breaks the old systems and why no known quantum algorithm breaks lattice problems. State clearly that your toy scheme is for learning only and must never be used to protect real data.

Go beyond: combine tracks or invent your own

The most impressive entries will not stop at one box. Here are starting points, from approachable to ambitious. None of them is required, and you are free to propose something entirely different.

Idea	Tracks combined	What makes it interesting
A logical qubit inside a QKD protocol	Steane code and BB84 or BBM92	Does encoding the transmitted qubits lower the QBER, and at what cost in qubits?
LDPC reconciliation in QKD post-processing	LDPC decoding and QKD	Replace a simple error-correction step with your own decoder, and plot final key rate against QBER
A neural-network decoder	QNN and error correction	Train a quantum or classical network to decode Steane syndromes, and compare with the lookup table
Logical qubits inside an optimiser	Steane code and QAOA or VQE	Does error protection improve the answer on a noisy simulator, or does the overhead cancel the gain?
Barren-plateau-aware chemistry	VQE and QNN mitigation	Apply identity-block initialisation or layer-wise training to a larger VQE ansatz
Quantum-safe key exchange pipeline	Lattice cryptography and QKD	Compare a lattice-based key exchange with QKD in security assumptions, cost and failure modes
QAOA for code design	QAOA and error correction	Search for good parity-check matrices or qubit layouts as an optimisation problem

Idea	Tracks combined	What makes it interesting
Decoder as a hardware design	LDPC decoding and FPGA or fixed-point design	Quantify the accuracy lost by low-precision arithmetic, as a first step toward hardware

Open invitation. If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early in the week.

How to scope a combined project. Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

What to submit and how it is judged

Every team submits a code repository or notebook, a short write-up (two to four pages), and a five-minute demo followed by questions. The scoring below applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up and in the live questions
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of demo and code	10	Readable, reproducible code and a clear five-minute story

Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

Submission checklist.

- ☐ Repository or notebook that runs from a clean start, with versions of the packages listed
- ☐ Write-up with the problem, method, results and an honest limits section
- ☐ At least one figure that shows the main result
- ☐ A short AI-use note: which tools you used, one thing the AI got wrong, and how you caught it
- ☐ A demo that every team member can take questions on

On using AI. AI assistants are welcome at every step. The AI-use note and the live questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.